

Module: `hangman`

This module implements a classic Hang-Man game together with a set of helper utilities for loading a word list, selecting a secret word, tracking player progress, and optionally providing “help” hints during gameplay. The functions are deliberately simple and are written for educational purposes, emphasizing clear logic over performance.

Functions

`load_words()`

Description

Loads a list of valid lowercase words from the file whose path is stored in the global constant `WORDLIST_FILENAME`. The function reads the first line of the file, splits it on whitespace, and returns the resulting list.

Parameters

None

Returns

- `list[str]` : a list containing all words read from the file.
(Typical size $\approx 55\,900$ items; loading may take a few seconds.)

Examples

```
words = load_words()          # → ['apple', 'banana', ..., 'zebra'] (~55 900 items)
print(len(words))             # → 55900
```

Edge Cases / Notes

- The function assumes the file exists and contains at least one line; any I/O error will propagate as an exception.
- It prints progress messages to stdout, which can be suppressed by redirecting `sys.stdout` if needed.

`choose_word(wordlist)`

Description

Selects and returns a single word at random from the supplied list.

Parameters

- `wordlist` (*list[Any]*): the source collection from which to pick a word. Elements are expected to be strings, but any type is technically allowed because `random.choice` will return whatever it receives.

Returns

- *Any*: the randomly chosen element from `wordlist`. When the list contains strings, a string is returned.

Examples

```
choose_word(['apple', 'banana', 'cherry'])  # → 'cherry' (random)
choose_word(['singleton'])                  # → 'singleton'
choose_word(['repeat', 'repeat', 'unique']) # → 'repeat'
```

```
choose_word(['good', 123, None])      # → None    # non-string element may be returned
choose_word([''])                     # → ''      # empty string is a valid element
```

Edge Cases / Notes

- **Empty list:** raises `IndexError: list index out of range`.
- The function does **not** validate that the elements are strings; passing non-string values can lead to surprising return types (e.g., `None`).

`has_player_won(secret_word, letters_guessed)`

Description

Determines whether the player has successfully guessed every distinct letter of the secret word.

Parameters

- `secret_word` (*str*): the target word, expected to be lowercase.
- `letters_guessed` (*list[str]*): collection of letters that have been guessed so far.

Returns

- `bool`: `True` if **all** characters in `secret_word` appear in `letters_guessed`; otherwise `False`. An empty `secret_word` is considered already “won”.

Examples

```
has_player_won('', [])                # → True
has_player_won('a', [])               # → False
has_player_won('a', ['a'])            # → True
has_player_won('apple', ['a', 'p', 'l', 'e']) # → True
has_player_won('apple', ['a', 'p', 'l'])    # → False
```

Edge Cases / Notes

- The function performs a linear scan; duplicate letters in `secret_word` do not affect the result.

`get_word_progress(secret_word, letters_guessed)`

Description

Creates a visual representation of the secret word showing guessed letters in their correct positions and underscores (`_`) for letters that have not yet been guessed.

Parameters

- `secret_word` (*str*): the word to reveal partially.
- `letters_guessed` (*list[str]*): letters that have been guessed so far.

Returns

- `str`: a string composed of correctly guessed letters and `_` placeholders for the rest.

Examples

```
get_word_progress('apple', [])        # → '_____'
get_word_progress('apple', ['a', 'p', 'l', 'e']) # → 'apple'
get_word_progress('banana', ['a'])     # → '_a_a_a'
get_word_progress('', ['a', 'b'])      # → ''
get_word_progress('test', ['t', 'x'])  # → 't__t'
```

Edge Cases / Notes

- The function always returns a string of length equal to `len(secret_word)`.
- No validation is performed on `letters_guessed`; non-alphabetic entries are simply ignored.

`get_available_letters(letters_guessed)`

Description

Returns a string containing all lowercase alphabet letters that have **not** yet been guessed, ordered alphabetically.

Parameters

- `letters_guessed` (*list[str]*): letters already guessed.

Returns

- `str`: concatenated alphabet letters still available for guessing.

Examples

```
get_available_letters([])           # → 'abcdefghijklmnopqrstuvwxyz'
get_available_letters(['a', 'b', 'c']) # → 'defghijklmnopqrstuvwxyz'
get_available_letters(list('abcdefghijklmnopqrstuvwxyz')) # → ''
get_available_letters(['a', 'a', 'b', 'c']) # → 'defghijklmnopqrstuvwxyz' # duplicates ignored
get_available_letters(['z', 'x', 'q', 'm']) # → 'abcdefghijklmnopqrstuvwxyz' (letters except m,q,x,z)
```

Edge Cases / Notes

- Duplicate entries in `letters_guessed` have no effect because the function checks membership with `in`.
- The function does **not** validate that entries are single lowercase letters; any non-matching entry is simply ignored.

`revealed(secret_word, available_word)`

Description

Selects a random letter from `secret_word` that is also present in `available_word`. This is used by the “help” feature (!) to reveal a missing letter.

Parameters

- `secret_word` (*str*): the target word.
- `available_word` (*str*): a string of letters that are currently *available* for revealing (typically the output of `get_available_letters`).

Returns

- `str`: a single character randomly chosen from the intersection of the two inputs.

Examples

```
revealed('apple', 'aeiou') # → 'e' (randomly chosen from {'a', 'e'})
revealed('banana', 'b')   # → 'b'
revealed('mississippi', 's') # → 's'
revealed('abc', 'abc')     # → 'a' # any of a,b,c could be returned
revealed('a1b2', '12')     # → '2' # works with digits as well
```

Edge Cases / Notes

- **No common letters** (e.g., `revealed('test', 'xyz')`) raises `ValueError: empty range for randrange() (0, 0, 0)`.
- **Empty `secret_word` or empty `available_word`** also raise the same `ValueError`.

- The function does not guard against these situations; callers should ensure that at least one common character exists before invoking.

`unique_letters(secret_word)`

Description

Counts the number of distinct characters in `secret_word`, preserving the order of first appearance.

Parameters

- `secret_word` (*str*): the word to analyze.

Returns

- `int`: the total number of unique characters.

Examples

```
unique_letters('')          # → 0
unique_letters('aaaa')      # → 1
unique_letters('abcde')     # → 5
unique_letters('abca')      # → 3
unique_letters('aAaA')      # → 2  # case-sensitive; 'a' and 'A' are distinct
```

Edge Cases / Notes

- The function treats uppercase and lowercase letters as different characters because it does not convert the input to a uniform case.

`hangman(secret_word, with_help)`

Description

Runs an interactive Hang-Man session in the console. The player has 10 guesses and may request a hint (the `!` character) if `with_help` is `True`. Incorrect consonants cost 1 guess, incorrect vowels cost 2 guesses, and each hint costs 3 guesses. The game ends when the player either guesses all letters or exhausts the guess count.

Parameters

- `secret_word` (*str*): the word the player must guess.
- `with_help` (*bool*): enables the hint (`!`) functionality when `True`.

Returns

None – the function interacts via `print` and `input` and terminates by printing a final win/lose message and, on a win, the calculated score.

Examples

Because the function is interactive, typical usage looks like:

```
hangman('python', with_help=True)
# → (prompts appear on the console; user inputs guesses)
```

Edge Cases / Notes

- The function relies on global imports (`random`, `string`) and the helper utilities defined in this module.
- Invalid inputs (non-alphabetic characters, repeated guesses, or a `!` when `with_help` is `False`) are handled with explanatory messages.
- If the player requests a hint without having at least 3 guesses left, a warning is printed and no hint is given.
- Scoring formula on win: `guesses_remaining + 4 * unique_letters(secret_word) + 3 *`

```
len(secret_word) .
```

- No explicit error handling is present for unexpected exceptions; any such error will abort the game.